F Y E O

FLARE

Repo: https://gitlab.com/flarenetwork/network/go-flare

Security Review Update: February 23, 2026

Reviewer: balthasar@gofyeo.com

Security Code Review FLARE

New security issues, 4

After the development team implemented the latest updates, FYEO conducted a review of the modifications. The primary goal of this evaluation was to ensure the continued robustness of the program's security features, safeguarding user funds and maintaining the overall integrity of the program.

General Updates:

This update integrates Avalanche's version 1.13 changes, which fundamentally rewrites the gas pricing mechanism for the Flare blockchain's EVM implementation. The core change implements a new dynamic gas limit and price discovery system that replaces the previous rolling window approach with a more sophisticated exponential pricing model. Under the new "Fortuna" upgrade, the base fee calculation shifts from a fixed window gas consumption tracker to a continuous state based system that maintains gas capacity, excess, and target parameters. The gas price now follows an exponential formula that adjusts more smoothly to network demand, while the gas limit becomes dynamic based on a target that can be adjusted by validators within bounded rates. This allows the network to scale capacity up or down in response to sustained demand changes, rather than being locked to fixed limits.

The implementation required extensive refactoring of the consensus engine and header validation logic, moving gas related calculations into a dedicated header package with separate modules for base fees, gas limits, dynamic fee windows, and the new ACP-176 state management. The code carefully maintains backward compatibility across multiple upgrade boundaries, with network-specific parameters for Flare, Songbird, and Costwo testnets. Critical changes include modifications to how block gas costs are calculated and verified, updates to the transaction pool's fee estimation logic, and changes to how the extra data field in block headers encodes fee state information - transitioning from a simple rolling window to a more complex state structure containing capacity, excess, and target excess values. The update also removes the concept of maximum base fee caps in favor of unbounded exponential growth, while introducing minimum gas price floors and sophisticated overflow handling to prevent arithmetic errors in extreme conditions.

Missing Error Handling in conflicts() Function

Finding ID: FYEO-FLARE-01
Severity: **Low**
Status: **Remediated**

Description

The `conflicts()` function at the end of the file performs a type assertion on `nextAncestorIntf` to `AtomicBlockContext` but only checks the `ok` boolean after the assertion. If the type assertion fails, the error message attempts to use `nextAncestor.ID()` which would be the zero value of the interface, potentially causing a panic or misleading error message.

Proof of Issue

File name: coreth/plugin/evm/atomic/import_tx.go
Line number: 479

```
nextAncestor, ok := nextAncestorIntf.(AtomicBlockContext)
if !ok {
    return fmt.Errorf("ancestor block %s had unexpected type %T",
nextAncestor.ID(), nextAncestorIntf)
}
```

Severity and Impact Summary

The error message construction uses `nextAncestor.ID()` when `nextAncestor` is the zero value of the interface (since the type assertion failed). This could cause a panic or produce a misleading error message with incorrect block ID information.

Recommendation

Use `nextAncestorIntf` in the error message instead of `nextAncestor`, or fetch the block ID before the type assertion fails.

Database Error Handling in Warp Verifier Backend

Finding ID: FYEO-FLARE-02
Severity: **Informational**
Status: **Remediated**

Description

The warp verifier backend now distinguishes between `database.ErrNotFound` and other errors when checking for known messages. However, if `GetMessage` returns an unexpected error (not ErrNotFound), the code wraps it with a parse error code, which may be misleading.

Proof of Issue

File name: coreth/warp/verifier_backend.go
Line number: 26

```
if _, err := b.GetMessage(messageID); err == nil {
    return nil
} else if err != database.ErrNotFound {
    return &common.AppError{
        Code:    ParseErrCode,
        Message: fmt.Sprintf("failed to get message %s: %s", messageID,
err.Error()),
    }
}
```

Severity and Impact Summary

Database errors (connection issues, corruption, …) are being reported as parse errors, which could mislead debugging efforts. The error code should reflect the actual failure type.

Recommendation

Use a more appropriate error code for database access failures, or create a new error code specifically for database errors to distinguish them from parse errors.

Incorrect error logging

Finding ID: FYEO-FLARE-03
Severity: **Informational**
Status: **Remediated**

Description

In the `executeTx` function, when a transaction is successfully executed, the debug log statement includes `zap.Error(err)`. At this point in the code flow, all error checks have passed and the function is about to return success. The `err` variable refers to the last error variable in scope, which could be nil or from a previous check. Logging an error in a success message is misleading and incorrect.

Proof of Issue

File name: avalanchego/vms/platformvm/block/builder/builder.go
Line number: 639

```
inputs.Union(txInputs)

backend.Ctx.Log.Debug("successfully executed transaction",
    zap.Stringer("txID", txID),
    zap.Error(err),
)
txDiff.AddTx(tx, status.Committed)
return true, txDiff.Apply(stateDiff)
```

Line number: 617

```
if inputs.Overlaps(txInputs) {
    // This log is a warn because the mempool should not have allowed this
    // transaction to be included.
    backend.Ctx.Log.Warn("transaction conflicts with prior transaction",
        zap.Stringer("txID", txID),
        zap.Error(err),
    )

    mempool.MarkDropped(txID, blockexecutor.ErrConflictingBlockTxs)
    return false, nil
}
```

Severity and Impact Summary

This is a logging issue that will either log nil or an unrelated error value in a success log message. It does not affect functionality but makes debugging confusing as successful operations appear to have errors in the logs.

Recommendation

Remove the `zap.Error(err)` line from the success log statement since there is no error in this code path.

Incorrect error logging in signature aggregation loop

Finding ID: FYEO-FLARE-04
Severity: **Informational**
Status: **Remediated**

Description

In the `AggregateSignatures` function, when processing results from the channel, the code logs errors using a variable `err` that is shadowed from the outer scope. The result contains `result.Err`, but the log statement references `err` which may be from a previous operation (the proto.Marshal call).

Similarly, when logging a duplicate signature drop, the code uses `zap.Error(err)` which refers to a shadowed variable from outer scope rather than any actual error related to the duplicate signature condition.

Proof of Issue

File name: avalanchego/network/p2p/acp118/aggregator.go
Line number: 127, 137

```
case result := <-results:
    if result.Err != nil {
        s.log.Debug(
            "dropping response",
            zap.Stringer("nodeID", result.NodeID),
            zap.Error(err),
        )
        continue
    }

if signerBitSet.Contains(result.Validator.Index) {
    s.log.Debug(
        "dropping duplicate signature",
        zap.Stringer("nodeID", result.NodeID),
        zap.Error(err),
    )
    continue
}
```

Severity and Impact Summary

The log statement uses `err` instead of `result.Err`, which means it's logging the wrong error or potentially a nil error. This makes debugging difficult as the actual error from the result is not logged. This is a logging issue where an incorrect error variable is referenced. Since this is a debug log for a non-error condition (duplicate signature), it may log nil or an unrelated error.

Recommendation

Change to `zap.Error(result.Err)`. Remove the `zap.Error(err)` line from the log statement since there's no actual error in the duplicate signature case.

Commit Hash Reference:

For transparency and reference, the security review was conducted on the specific commit hash for the FLARE repository. The commit hash for the reviewed versions is as follows:

03e6902055e5f484e0adcadcb559dc408fb910ab

Remediations were provided with the commit hash: 3edd7f8ce2519dccbd3748b2396417d1d5f17d30

Conclusion:

In conclusion, the security aspects of the FLARE program remain robust and unaffected by the recent updates. Users can confidently interact with the protocol, assured that their funds are well-protected. The commitment to security exhibited by the development team is commendable, and we appreciate the ongoing efforts to prioritize the safeguarding of user assets.